

面向多智能体协作的 低开销通信、状态传递与共享记忆机制

目录

CONTENTS

01

项目介绍

02

项目亮点

03

技术难点

04

项目测试

05

作品演示

Part 01 项目介绍



1.1 项目背景



当前大模型能力发展提升，但多是云端部署，**隐私安全**得不到保障
本地**算力受限**情况下只能部署**参数量低的模型**，如何**更好的发挥模型能力**

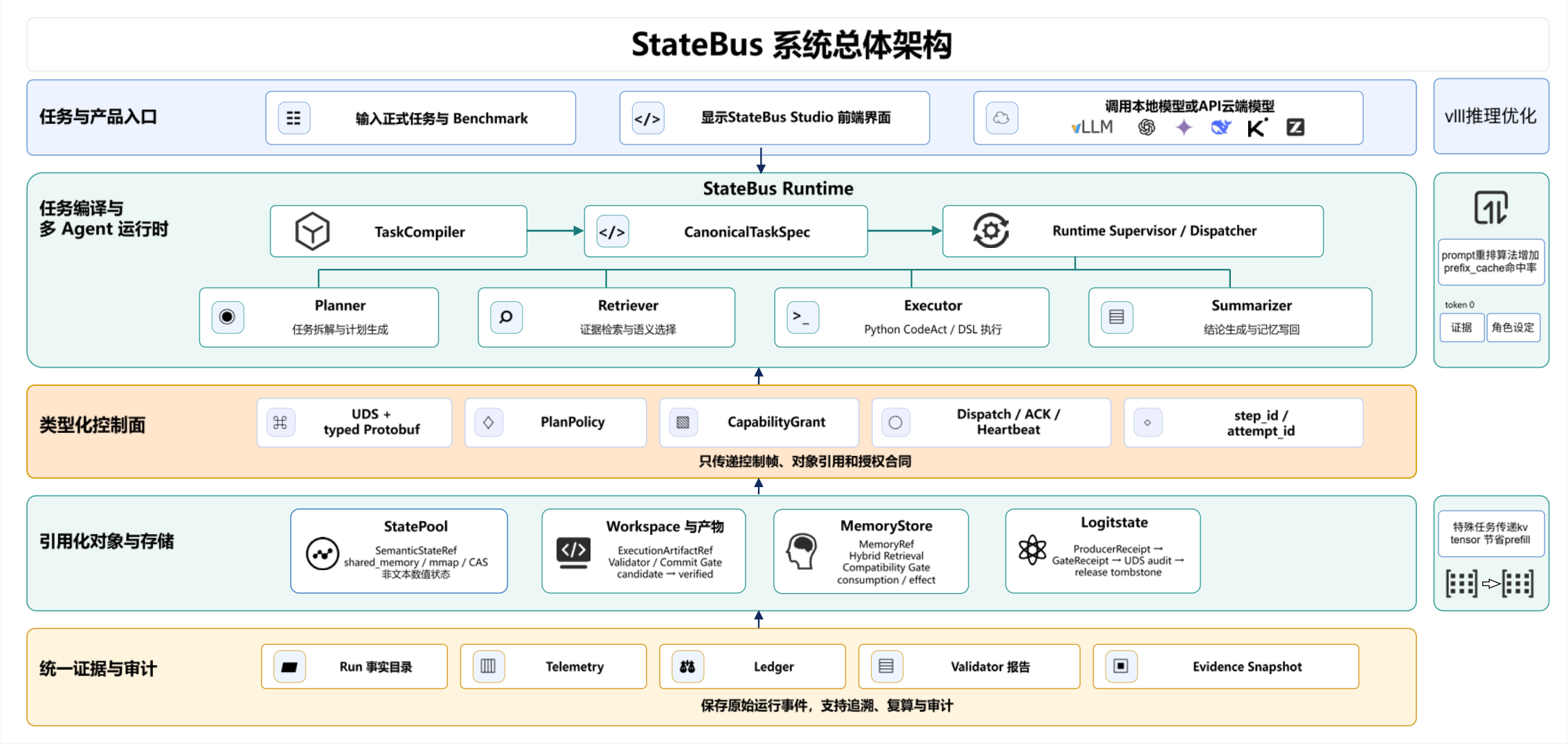
现实约束



设计选择



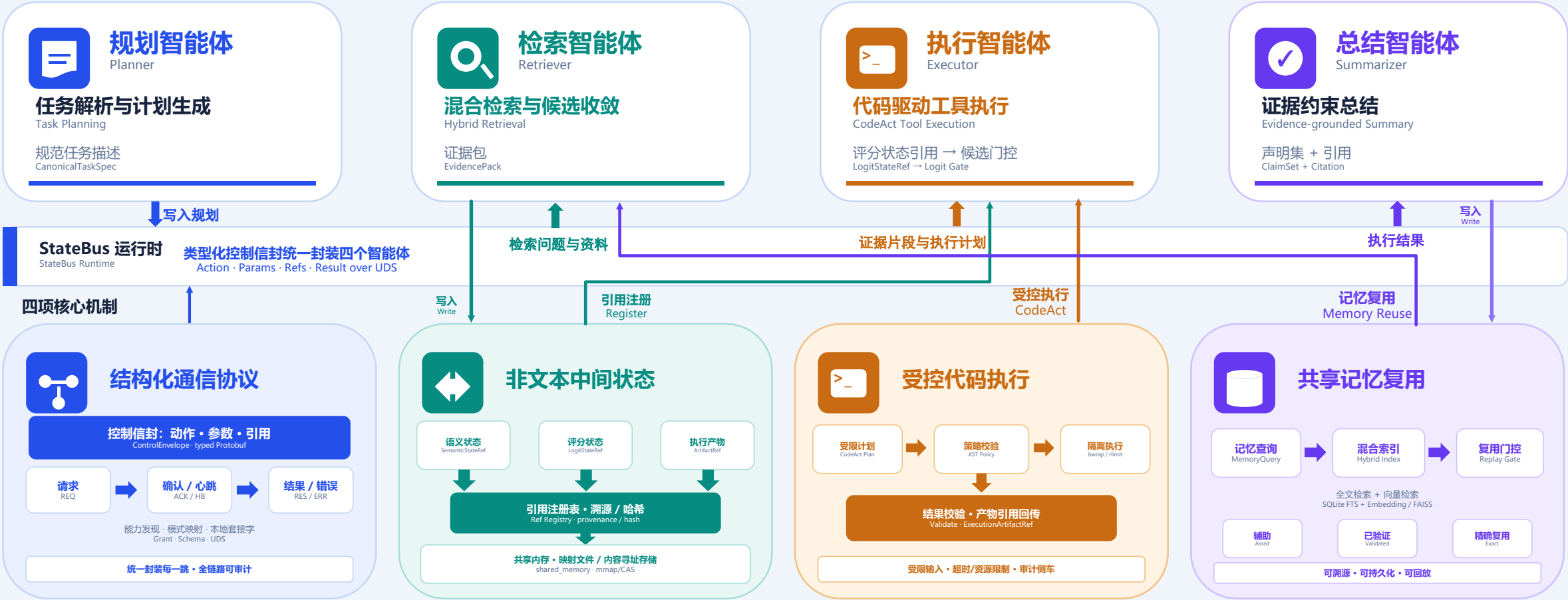
1.2 StateBus 系统总体架构





1.3 项目总览——各模块关系图

多智能体协作主干



结构化通信 · 非文本状态直传 · 受控代码执行 · 跨任务记忆复用 · 全链路审计

1.4 赛题完成情况

赛题要求 7 项全部覆盖

赛题要求	完成情况	亮点
≥3 个 Agent 协同	Planner / Retriever / Executor / Summarizer	4 Agent
结构化通信协议	AgentMessage	结构化，可追踪
非文本状态传递	非文本状态传递机制 Embedding 、 Logitstate 、 KV	减少 token开销、减少 prefill / decode
共享记忆模块	Qdrant向量数据库 + Dense相似度匹配/BM25关键词匹配 + Planner 校验	可检索、校验、复用
开源 OS 开发测试	openEuler	满足要求
两组连续任务验证	超三组多轮任务	超额完成
性能对比数据	Token消耗数 / time / hit rate / accuracy	多维评估

Part 02 项目亮点



2.1 结构化通信：从“能传”到“可控制”

相关工作：结构越来越清晰，但控制仍留给系统

四类代表方案分别解决消息协作、路由状态、工具协议与互操作

	AutoGen / AgentChat (2023) 自然语言协作	缺口 任务边界留给提示词
	LangGraph Handoffs (2024) Command 路由 • State 更新	缺口 能传递，难约束
	MCP (2024/2025) JSON-RPC 工具 / 资源协议	缺口 偏模型—工具
	A2A Protocol (2025) AgentCard • Task • Message	缺口 预算、修复依赖系统

抽取稳定
控制字段

StateBus：把消息升级为受约束控制合同

稳定字段明确描述任务、能力、输入输出和预算风险

		
身份与能力 task_id / task hash capability_id / allowed tools	输入与输出 input refs / schema required fields / output schema	风险与修复 budget / risk precheck reject / replan / repair
执行控制结果 可拒绝 • 可校验 • 可修复 不匹配即停止 schema / 依赖 / 证据可核验 局部 replan / repair		

关键差异

结构化通信不只要“理解提示词”，更要执行前可校验、失败后可修复

已有工作解决“消息格式、路由与互操作”；StateBus 进一步补齐能力边界、预算风险与错误闭环。



2.1 结构化通信协议：StateBus 最小控制合同



Planner 只提出方案；Policy 绑定任务、角色、输入、输出与预算，再为每个角色生成最小 RoleView



最小暴露

同一任务合同投影为不同 RoleView：只读获准对象，只交付约定产物

Planner 负责提出方案；Policy 决定是否授权、如何收窄，以及失败后进入重规划还是拒绝。



2.2 StateBus 非文本状态：三类状态，各解一个瓶颈



公开方案分别解决检索、解码控制或前缀缓存；StateBus 进一步补齐跨进程状态契约、授权与生命周期证据。

论文 / 开源项目的典型边界

“未覆盖”指其公开机制并非为 StateBus 的运行时证据目标设计

1 RAG / 向量数据库 [1]

已解决：稠密向量索引按相似度召回

未覆盖：跨 PID 消费、行为影响与释放回执

2 Transformers LogitsProcessor [2]

已解决：生成步骤内读取并修改 logits

未覆盖：跨 Agent 的 Ref、lease 与执行授权回执

3 vLLM APC / SGLang RadixAttention [3][4]

已解决：相同 token prefix 自动复用 KV

未覆盖：显式角色交接、任务绑定与业务级消费证明



StateBus：三类状态解决三类缺口

状态对象 = payload + contract + Ref + receipt + release

E

Embedding / SemanticStateRef

证据筛选可传递、可验证

float32 query×candidate matrix → 独立 PID cosine top-k → selected IDs

9/9 跨 PID · Token ↓47.4% · wire ↓64.85% · 质量 10/10

L

LogitStateRef / Retry Gate

把不确定选择变成执行授权

candidate probability + other_mass → top-1 / margin gate → retry 或 fail closed

Validator 8/12→12/12 · 歧义 3/5→5/5 · 错误放行 2→0 · 19/19 释放

E

Engine-local KV continuation

把已计算前缀交给下游角色

Worker-local handle + suffix → 恢复 4096-token KV → one-shot release

computed prefill ↓85.22% · TTFT ↓61.62% · 主链 wall ↓5.69% · 10/10 正向

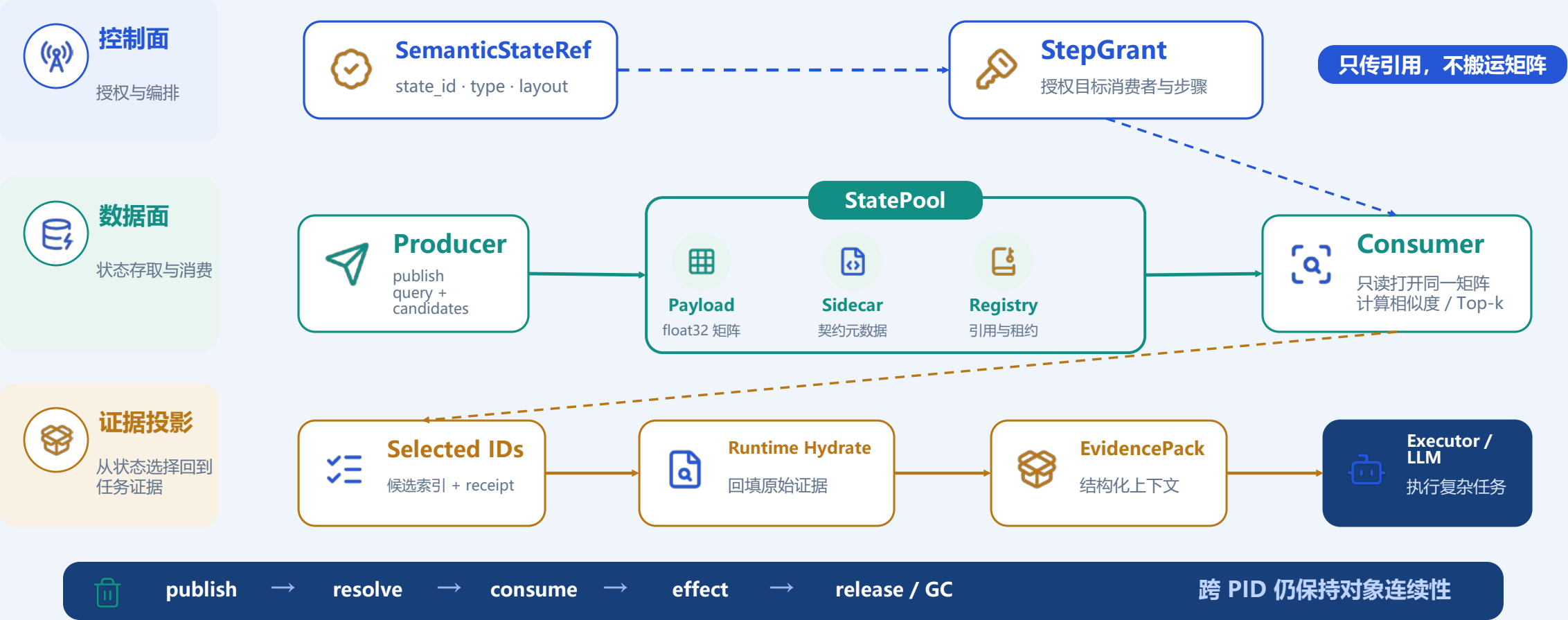
三类状态不是三个“缓存”：Embedding 选证据，Logit 授权执行，KV 继承计算



2.2 非文本状态1: embedding 跨进程流转



控制面只流转轻量引用；数据面保存连续载荷；证据面把选择结果还原为可执行上下文。



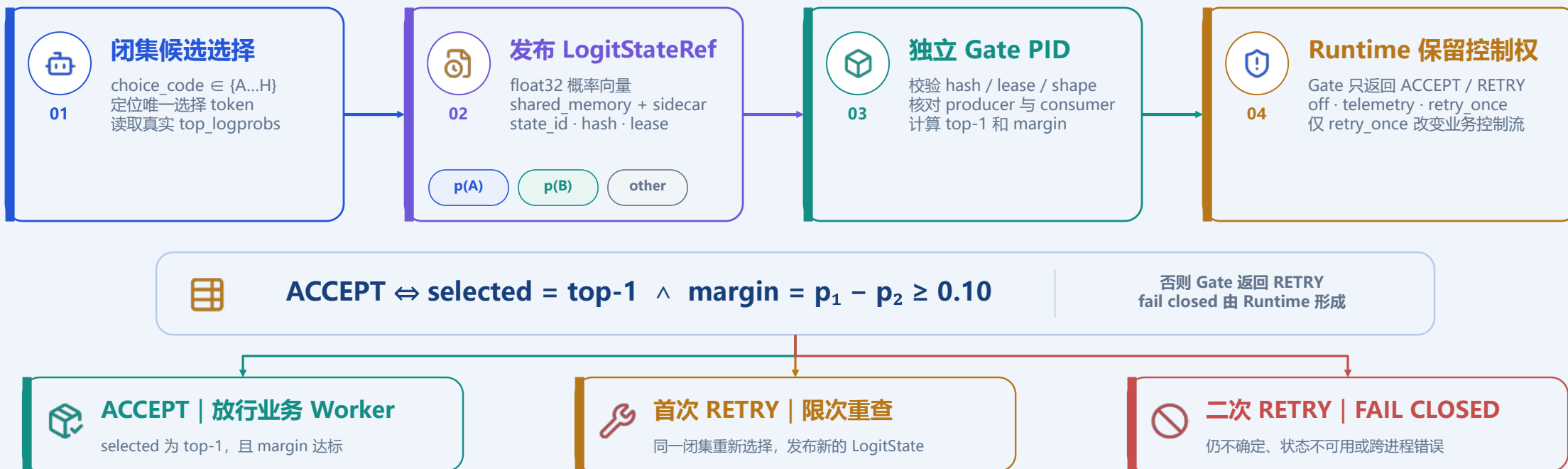


2.2 非文本状态2: Logit Retry Gate 让概率直接约束执行

真实 token 概率不是模型自报 confidence; 它以短生命周期状态进入独立授权边界

非文本状态闭环

Executor 选择之后、CodeAct / DSL / 业务 Worker 启动之前



审计闭环: ProducerReceipt → GateReceipt → UDS audit → release tombstone | 受控挑战 19/19 跨 PID 消费并释放

核心价值

生成权 ≠ 授权权: 模型提出候选, 独立 Gate 计算规则, Runtime 决定是否执行



2.2非文本状态3：KV显式继承



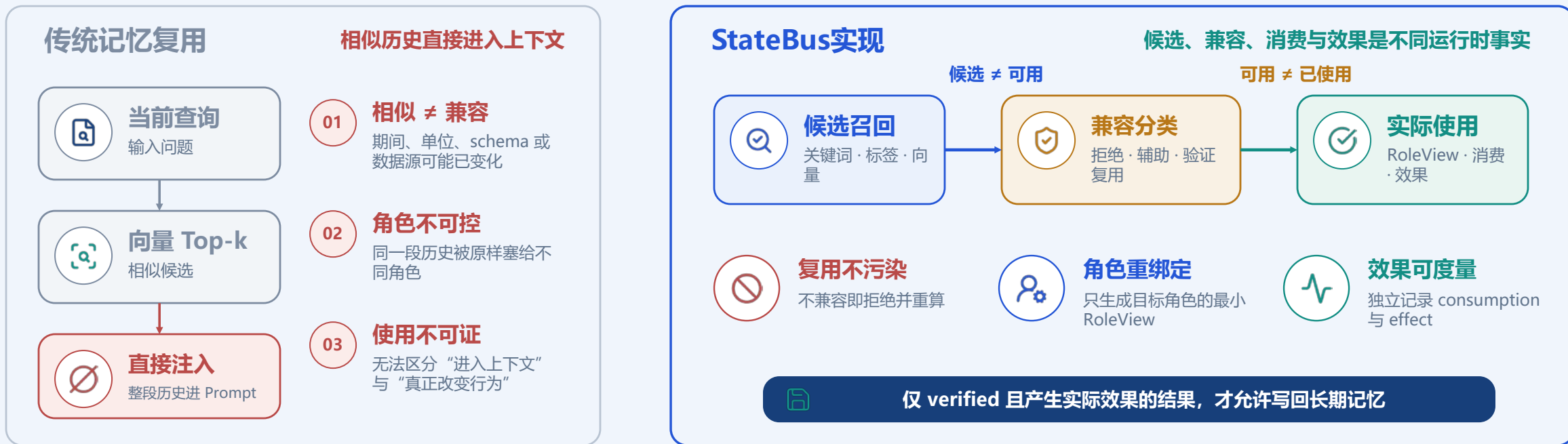
同一任务的 Executor→Summarizer 角色边，通过 capture、绑定、load、release 四步交接已计算前缀。





2.3 记忆复用：先验证再使用

传统方案只能证明“检索到相似历史”；StateBus 进一步回答“能不能用、谁能用、是否真的产生效果”。



对比结论

传统方案复用“相似文本”；我们的实现复用“经过兼容与效果验证的可信对象”



2.3 记忆复用：召回公式、兼容门与实际使用证据



RRF 只负责稳定排序；Memory Passport 决定准入；RoleView、消费回执和效果记录证明真实复用。



机制边界

RRF 解决 “先看谁”，兼容门解决 “能不能用”，消费与效果记录证明 “是否真的用了”



2.4 CodeAct: 代码可以生成，但只有验证产物能够离开工作区



同一授权入口，两种执行表示，统一汇入产物与质量门

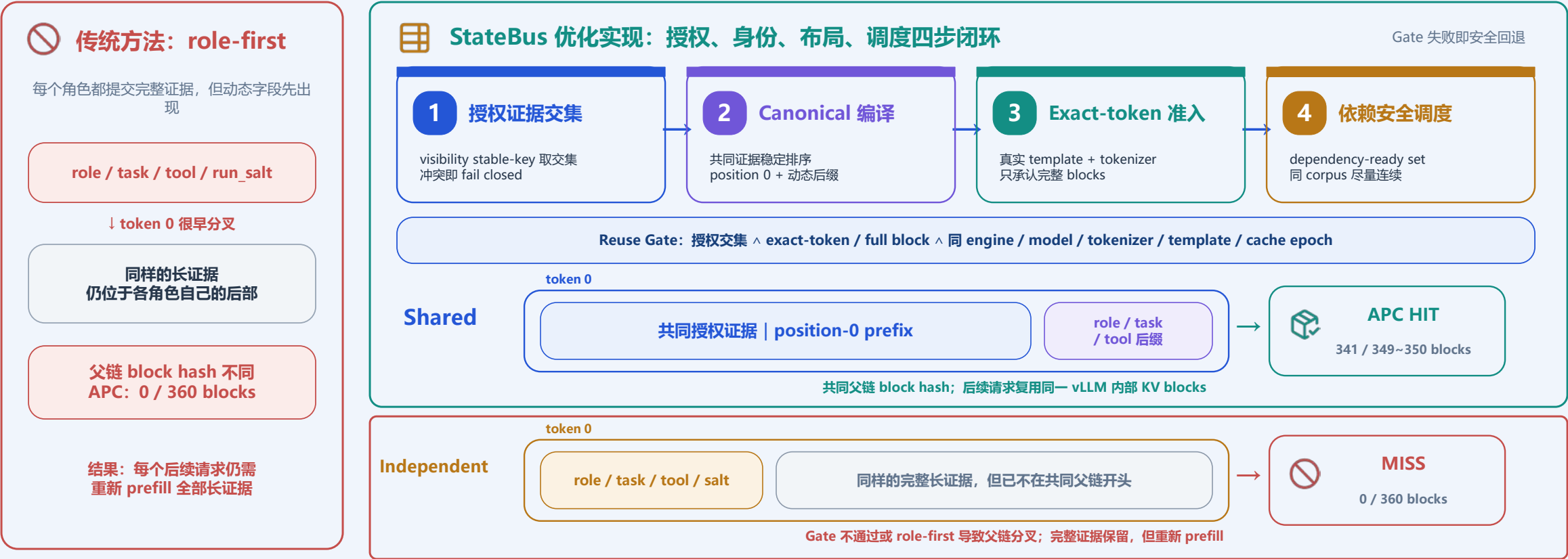


核心边界

执行表示可以不同，但授权、验证与结算一致——只有 verified ArtifactRef 能进入下游

2.5 基于前缀缓存命中的本地推理加速优化

传统 role-first 每次重复 prefill; StateBus 先判定复用资格, 再提交 Shared 布局



Part 03 技术难点



5.1 输入可信：对象能传，不代表当前任务能用



难点 A 可靠传递 | 控制语义和数值状态同时存在

自然语言难稳定表达角色、能力和合同；数值状态若文本化，又会丢失 dtype、shape、encoder 与来源链。



A 解决：双平面 + 三层授权 + 消费闭环

UDS / Protobuf 传控制；SemanticStateRef 留数值对象；PlanPolicy → Grant → 发送前复核；消费回执再关联 effect、release 与 GC。

证明身份、授权、结构和真实使用

难点 B 共享记忆 | 相似内容可能是错误证据

任务意图、字段、对象清单、lineage、runtime 或角色权限一处变化，历史结论就可能失效。



B 解决：候选 → 兼容 → 投影 → 效果 → 提交

混合检索只生成 candidate；Compatibility Gate 检查硬条件；RoleView 最小可见；consumption 与 effect 分开；仅 verified + effective 才提交。

相似度不能覆盖结构与来源冲突

先证明“这个对象是谁、当前是否获准使用”，再讨论它是否值得复用。



5.2 执行可信：程序跑完，也可能不安全、不正确、不可恢复



风险 01 系统调用面无法靠 AST 覆盖

路径、网络、进程、文件系统和资源滥用，必须由操作系统边界收口。

风险 02 业务正确性不由退出码保证

schema、单位、数值范围、事实、来源和引用关系都可能错误。

风险 03 重试会产生两个并存的执行世界

旧 Worker 晚到成功、旧候选和旧 Grant 都可能污染新的 attempt。

四条证明链把执行结果提升为可信产物

- 01 运行安全 | Grant + AST / 路径 Policy + 非 root bwrap + 资源限制
- 02 业务质量 | Validator 检查结构、事实、来源，并独立复算关键结果
- 03 受控修复 | 新源码 hash + 新 workspace，重新走全部安全与质量门
- 04 恢复审计 | 稳定 step + 新 attempt / Grant；拒绝晚到结果；Run 统一证据

能运行 · 可相信 · 可恢复 · 可追溯

StateBus 不以“修好代码”为由扩大权限，也不让“重试成功”覆盖旧执行事实。



5.3 缓存可信：命中率不是第一目标，正确继承才是



PREFIX

横向复用 | 命中取决于精确 Token Block

共同内容必须从 position 0 开始；空白、顺序、模板或 tokenizer 漂移都会 miss；缓存状态由 engine 管理。



P

确定性前缀编译 + 兼容签名 + 命中证据

固定共享证据位置与模板；前缀合同和 engine 消费事件分别记录；完整 Prompt 保持发送；未命中直接 full prefill。

复用的是 prefill，不省略任务输入

KV

纵向续接 | 多层神经状态必须完全兼容

Token、model、engine、generation 与 layout 必须一致；scheduler 接受不等于 Worker forward 真正继承。



K

engine-local handle + 状态机 + 双重 proof

强绑定 task / step / attempt / token digest / runtime；READY 后 one-shot 消费；scheduler + Worker 双 proof；终态 finally release。

任何签名、状态或 proof 缺失都算 miss

路由优先级：READY 显式 KV → Prefix / APC → 完整 Prefill；失败不伪装命中，收益不重复计算。

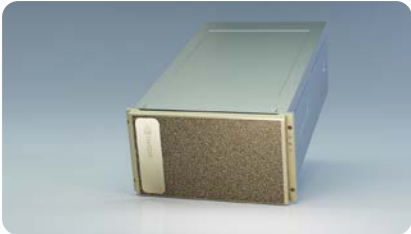
Part 04 项目测试



4.1 实验设置



实验的硬件软件环境、所选模型、任务设置



NVIDIA DGX A100 官方产品图



硬件环境

NVIDIA A100 80GB PCIe × 1
Driver 565.57.01 · CUDA 12.7
显存 81920 MiB · 单卡测试



E2 | 两组连续任务链 · 30 轮

30+

连续任务

运营任务链10 轮

财务任务链 10 轮

长文档 报告处理10 轮

CSV: schema / 缺失率、聚合 / 异常、drift / 清洗、lineage

财报: 抽取、差值、跨公司对比、字段别名、记忆拒绝、趋势汇总

2 份离线运营报告 × 5 个三季度指标



系统环境

openEuler 24.03 LTS SP3
NVIDIA-SMI 565.57.01
Compute Mode: Default



模型配置

Qwen3-32B
同一模型 · 同一单卡设置
聚焦通信与状态机制差异



E5 | 五类独立能力 · 25 case 测试agent能力泛化性



8
财报抽取



5
多期趋势



5
跨表关联



4
条件聚合



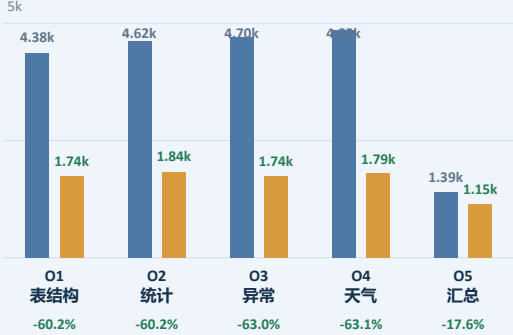
3
异常清洗

4.2 总体实验结果：验证所有机制总体效果

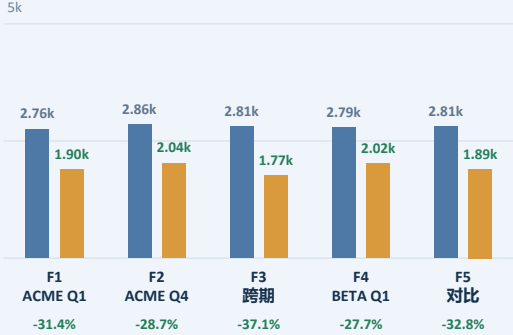


Total Token 深色 Prompt / 浅色 Completion 0-5k

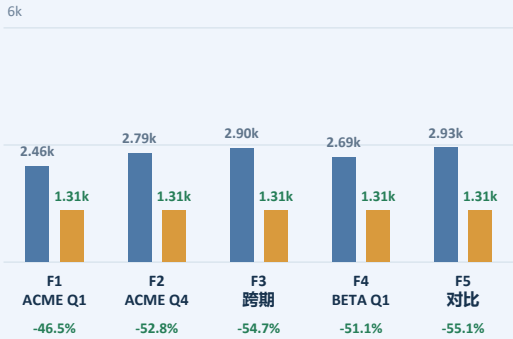
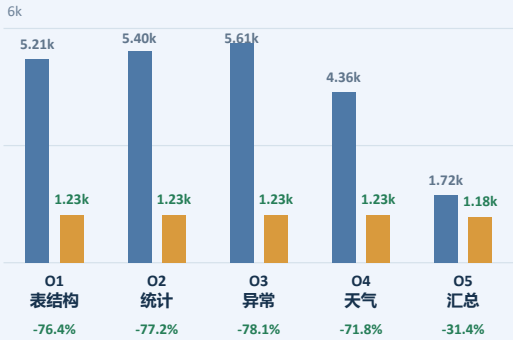
Operating · 病例 / 天气表分析 · 5 个连续任务



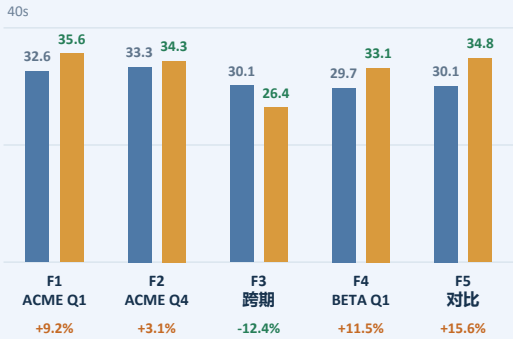
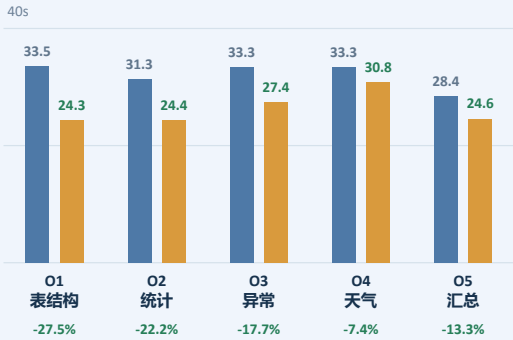
Financial · 跨期财报分析 · 5 个连续任务



wire bytes 0-6k B



任务耗时 L0 L3 0-40 s



完整系统总体效果

通信与执行开销 均实现下降

Total Token ↓ 47.40%

33,974 → 17,870

Wire Bytes ↓ 64.85%

36,069 → 12,677 B

端到端耗时 ↓ 6.32%

315.68 → 295.73 s

10/10 任务通过
50/50 逻辑消息保持不变

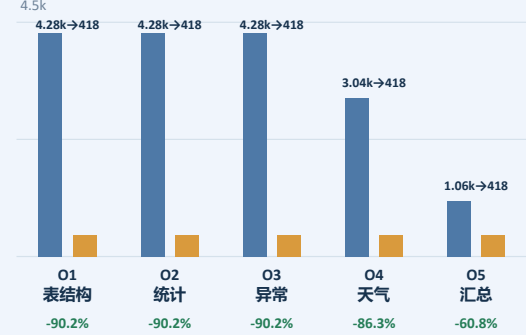
结论：完整系统显著降低 Token 与传输负担，任务质量不降，端到端耗时同步改善。

4.3 结构化通信消融实现

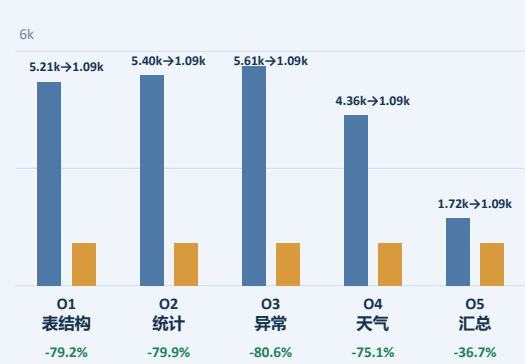


control bytes0—4.5k B

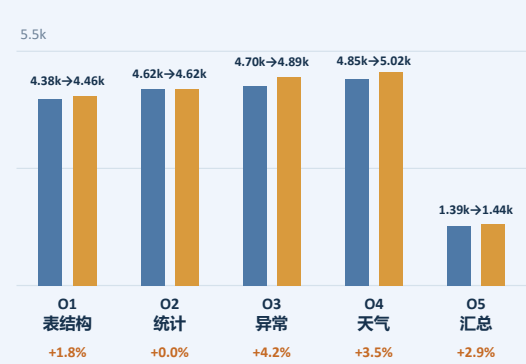
Operating · 病例 / 天气表分析 · 5 个连续任务



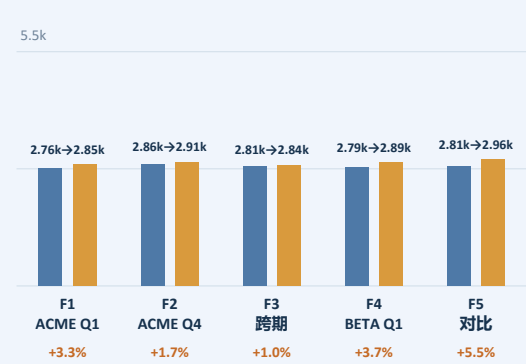
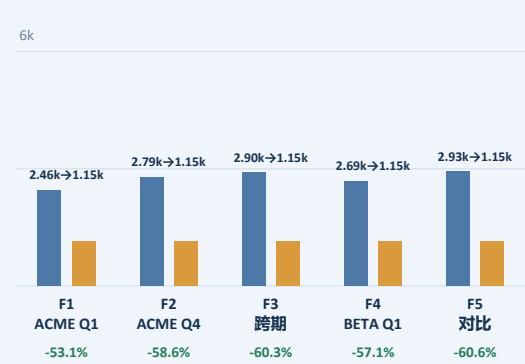
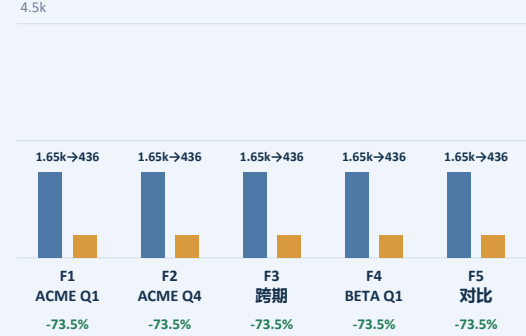
wire bytes0—6k B



Total TokenL0L10—5.5k



Financial · 跨期财报分析 · 5 个连续任务



结构化通信消融

两类通信负载
同步显著下降

Control Bytes ↓ 83.05%

25,196 → 4,270 B

Wire Bytes ↓ 68.95%

36,069 → 11,200 B

4.4 非文本状态消融实验

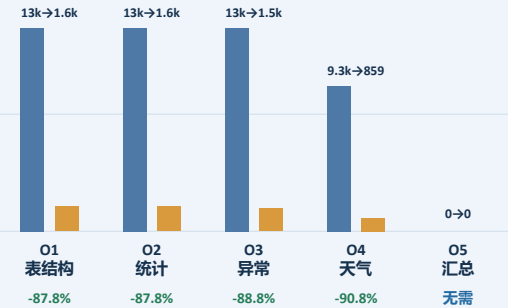


raw evidence bytes

0–15k B

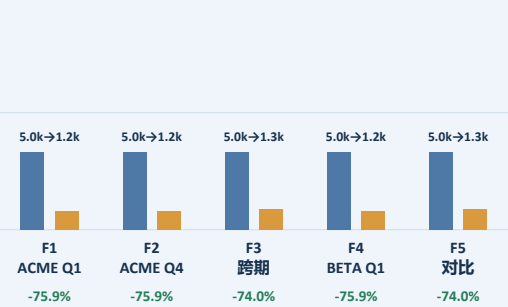
Operating · 病例 / 天气表分析 · 5 个连续任务

15k



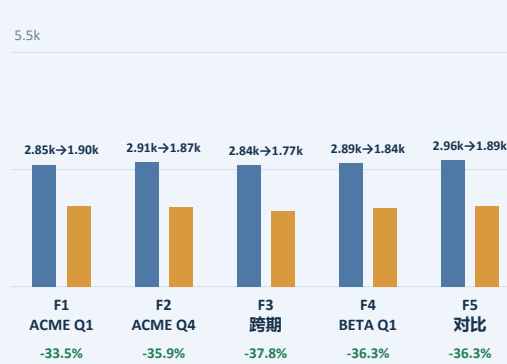
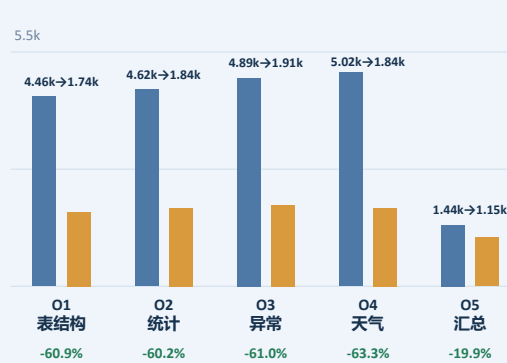
Financial · 跨期财报分析 · 5 个连续任务

15k



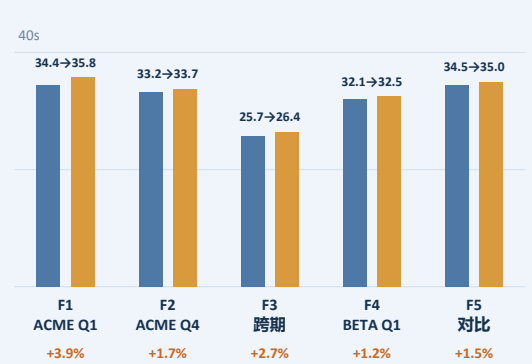
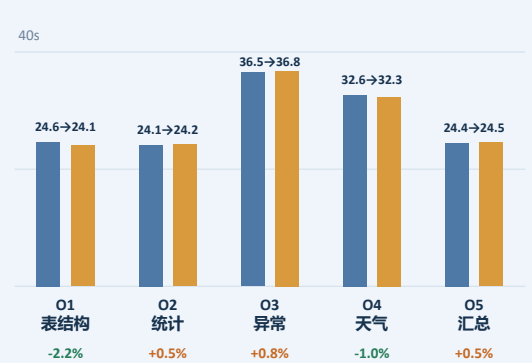
Total Token

0–5.5k



任务耗时

L1 L2 0–40 s



非文本状态消融

证据与 Token 同步大幅压缩

Raw Evidence ↓ 84.04%

73,266 → 11,693 B

Total Token ↓ 49.16%

34,891 → 17,739



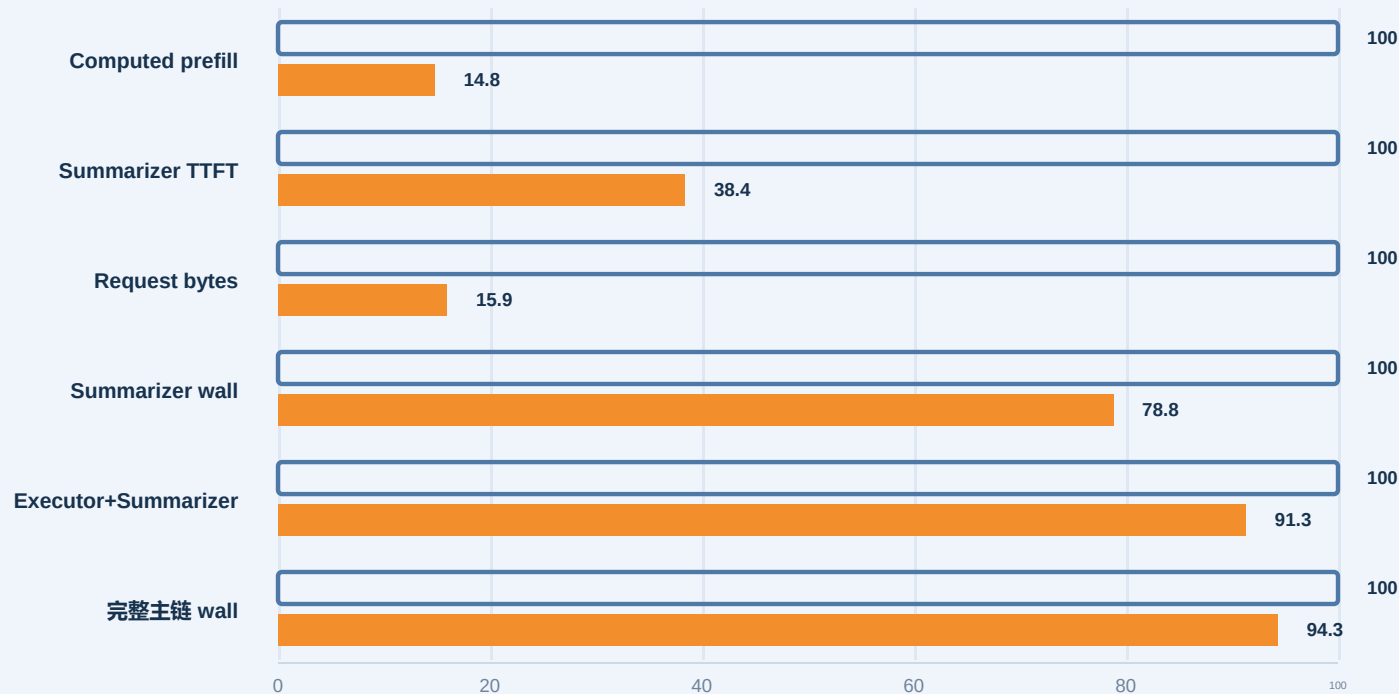
4.5 KV 非文本状态实验：显式继承带来端到端净收益



Qwen3-32B · 10 个任务 / 20 次计量执行 · APC=false · 纵轴以未启用 KV 的开销归一化为 100

关键开销对比 (Baseline = 100)

■ 未启用 KV ■ 启用显式 KV



所有下降指标均获得稳定改善

10/10 任务为正向收益 · A/B 质量等价 10/10

Computed prefill	↓ 85.22%
Summarizer TTFT	↓ 61.62%
Request bytes	↓ 84.07%
Summarizer wall	↓ 21.22%
Executor + Summarizer	↓ 8.70%
完整主链 wall	↓ 5.69%

捕获成本

Producer wall ↑6.39%

store 1.713 s · load 0.297 s · 单 handle 1 GiB

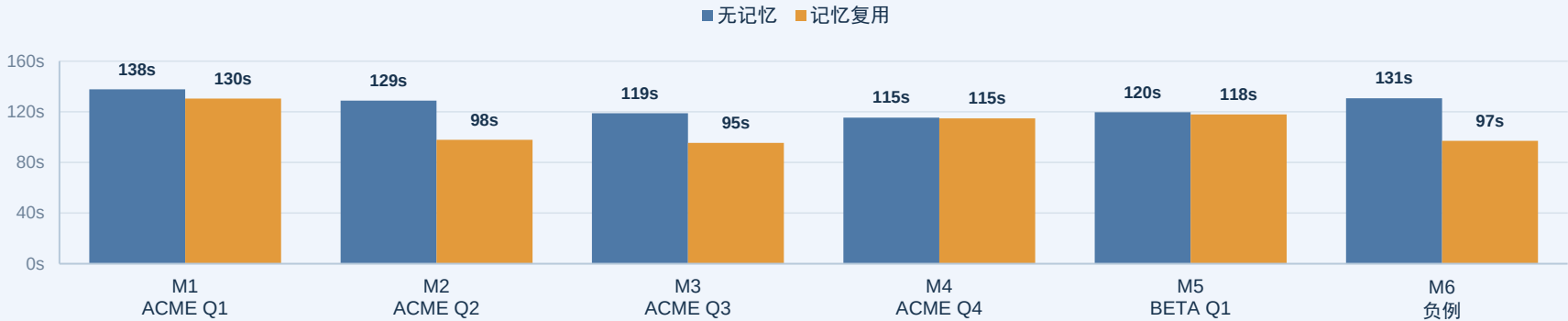
显式 KV 的 Consumer 节省覆盖捕获成本：在保证质量不变的同时获得端到端净收益



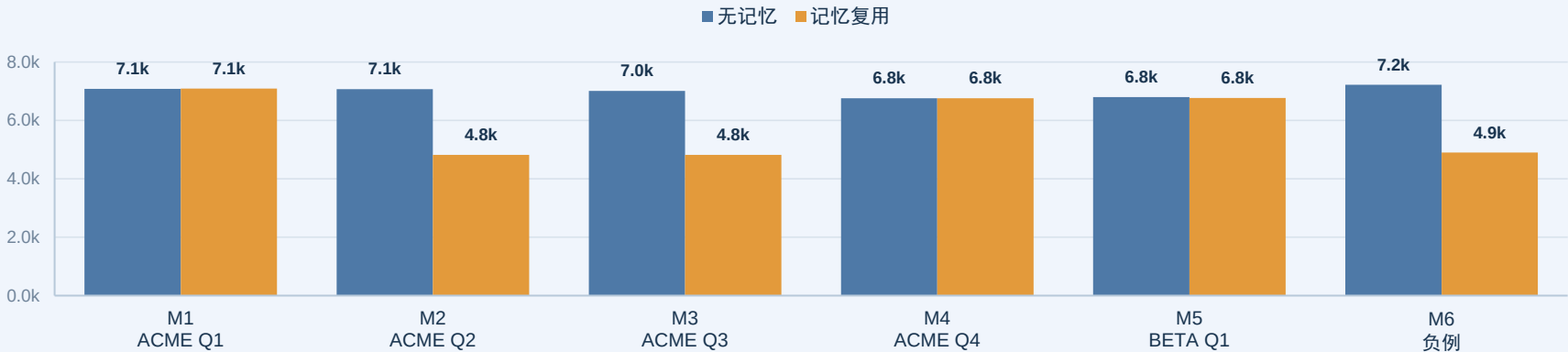
4.6 记忆复用消融实验



任务耗时 (s)



Total Token (k)



有效复用降低执行开销

任务耗时 ↓ **18.49%**
516.1 → 420.7 s

Total Token ↓ **23.75%**
28,379 → 21,638



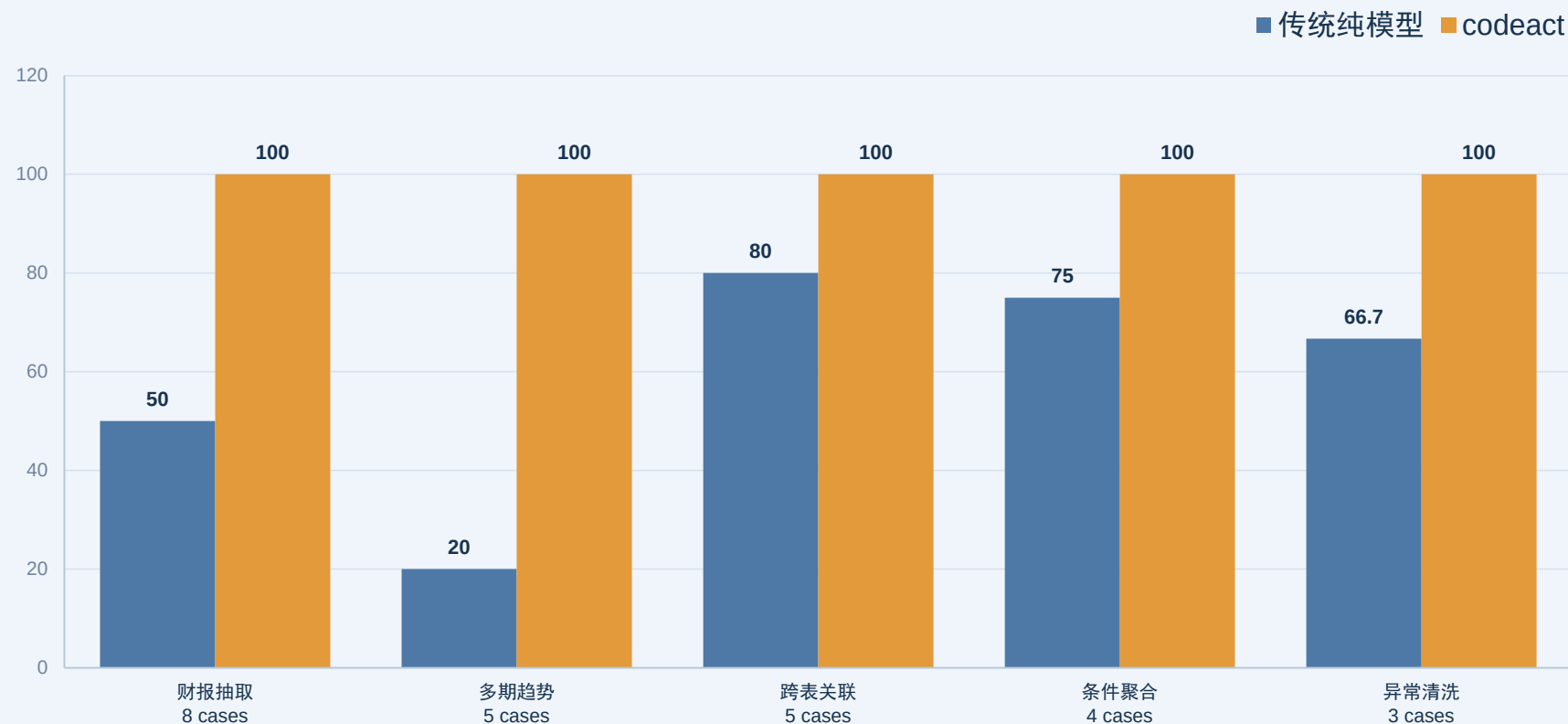
20 次任务查询 → 7 次实际复用;
39 / 48 个不兼容候选被拒绝。



4.7 CodeAct 能力消融实验



五类任务正确率 (%)



五类任务全部通过

总体正确率

56% → 100%
↑44 pp

14 / 25 → 25 / 25 (+11 cases)

逐类正确率提升 (全部列出)

财报抽取	↑ 50.0 pp
多期趋势	↑ 80.0 pp
跨表关联	↑ 20.0 pp
条件聚合	↑ 25.0 pp
异常清洗	↑ 33.3 pp



4.8 本地推理前缀缓存加速：分任务实验



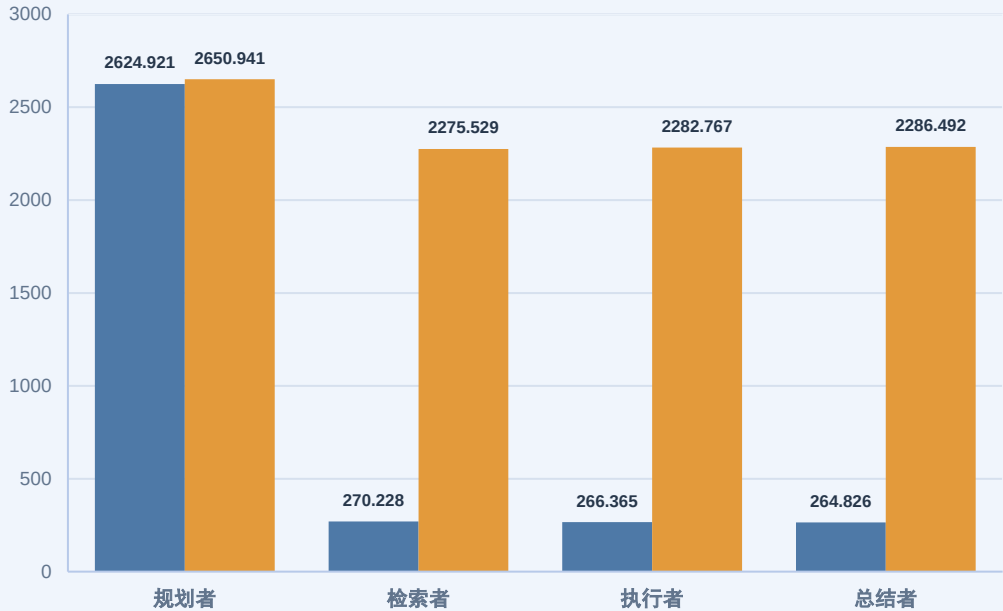
四组交替顺序、40 请求；每个柱为同一角色在 4 个 pair 中的均值，统计包含全部任务

■ 启用优化 ■ 未启用

同模型 · 同证据 · 同生成参数 · 40/40 输出合同通过

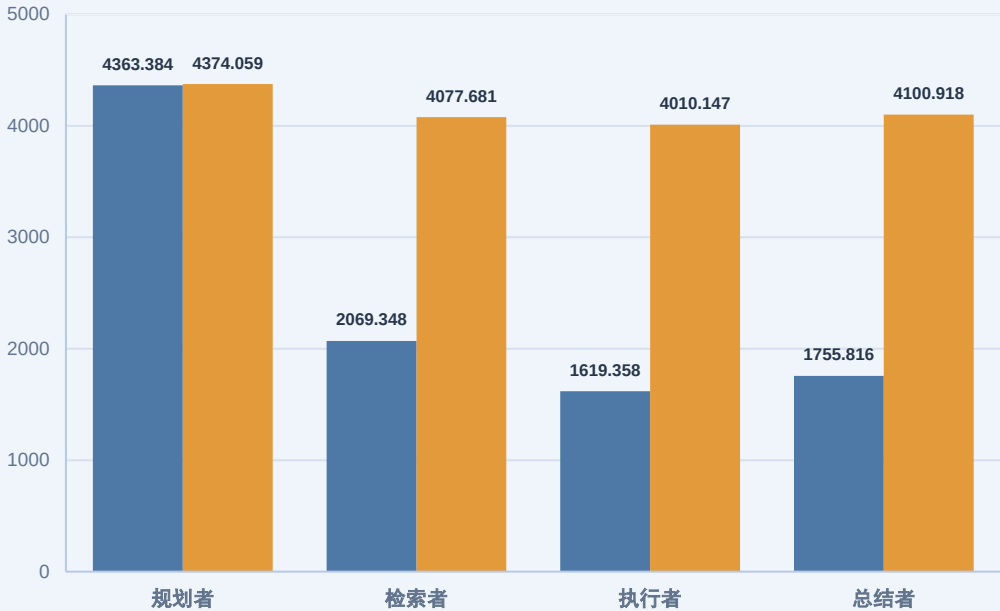
各任务 TTFT (ms)

规划任务近似不变，其余任务显著下降



各任务端到端时间 (ms)

prefill 消除后，decode 仍需执行

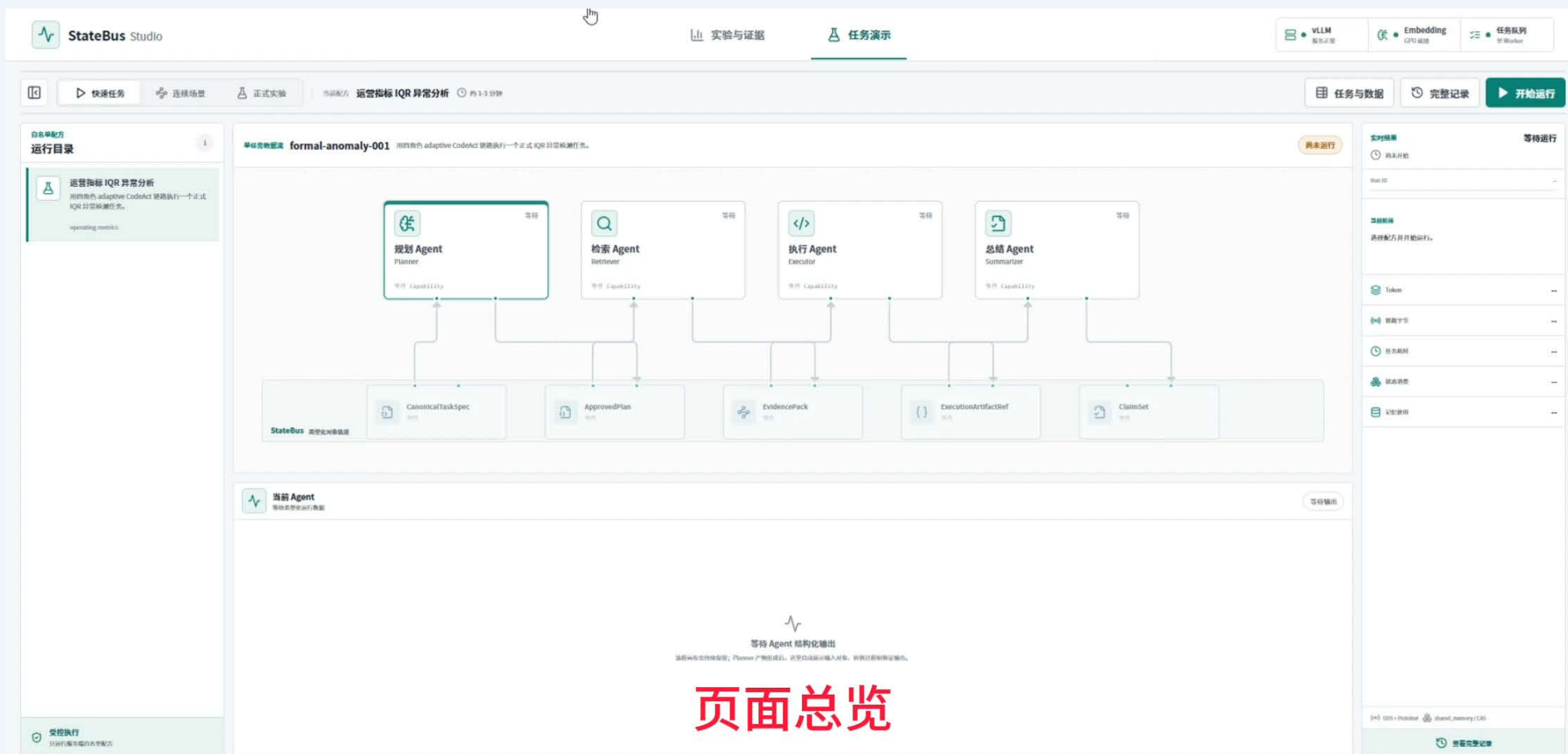


总体提升

前缀缓存命中率提升到 **78.02%** | 平均 TTFT 2357 → 738 ms (**-68.7%**) | 平均端到端 4117 → 2345 ms (**-43.0%**)

Part 05 作品演示

5.1 作品演示播放



谢谢

参赛团队 SynapseX

完成日期 2026.7.7